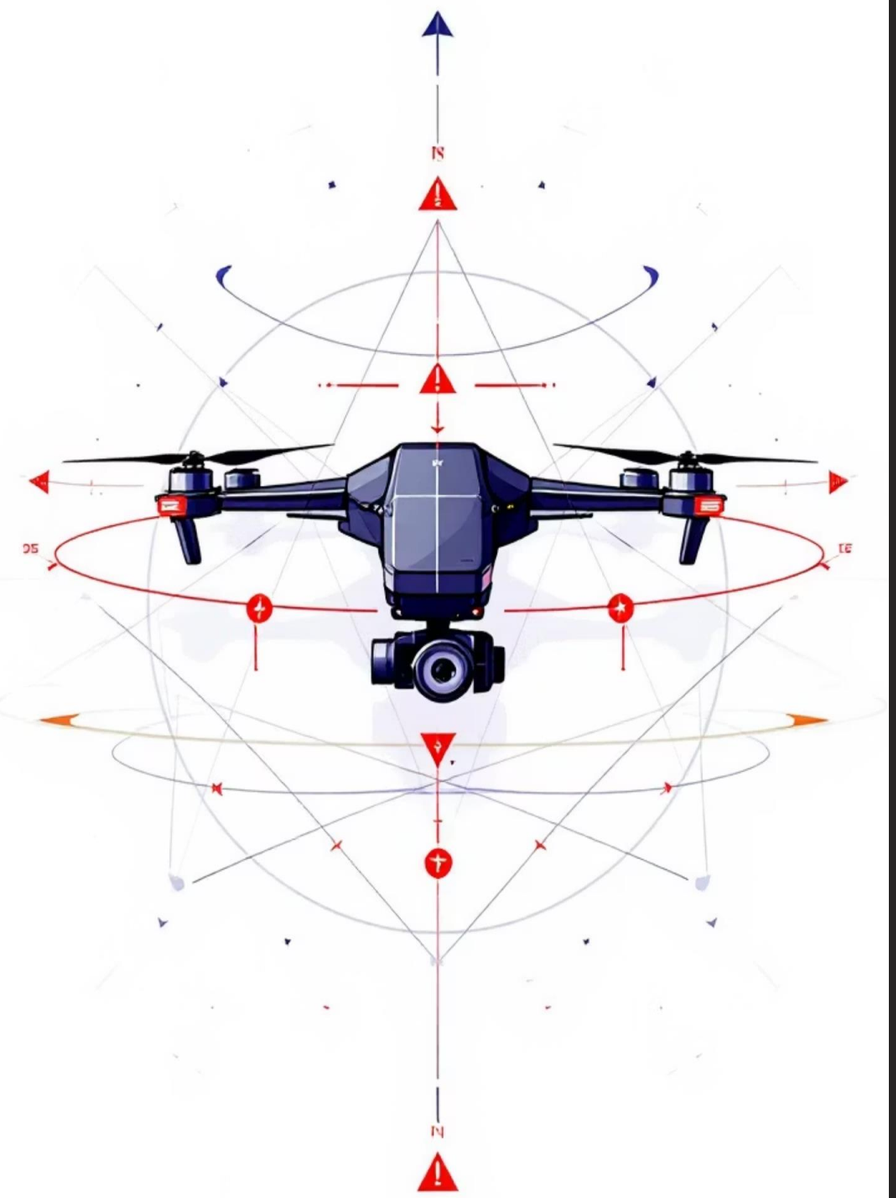


Autonomous Drone Landing on Moving Platform using Reinforcement Learning

Students : Youssef MIRI, Adrien WAELES-DEVAUX, Nicolas CONSALVI

Supervisors : Dr Jean MARTINET, Mariia BERDNYK

Project Overview

Objective

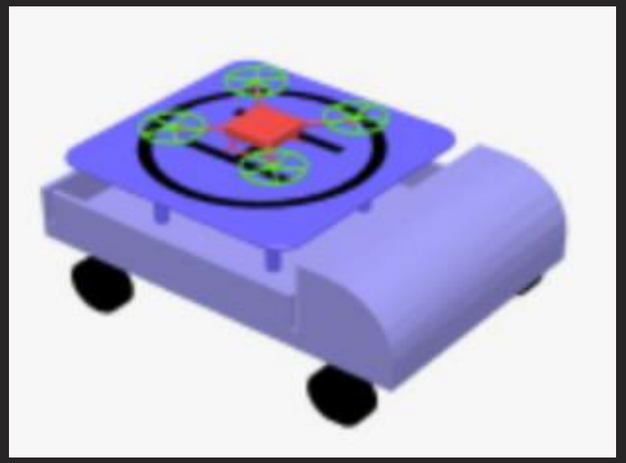
Train a quadrotor to land precisely on a moving vehicle.

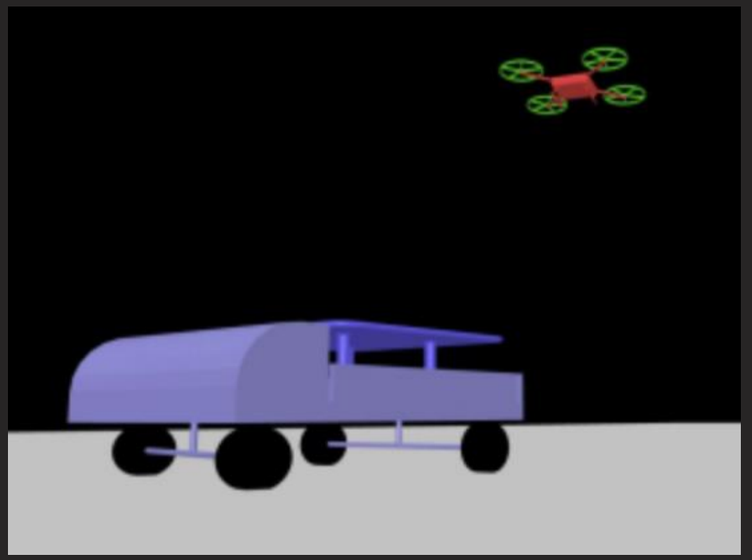
Core Challenge

Synchronize 3D flight with variable speed and lateral "sway" target.

Frameworks

Gymnasium, MuJoCo (Physics), NumPy.





Observation Space: Input Features

Agent decisions based on an 11-dimensional vector.

Previous Actions (4)

Maintain flight continuity.

Pitch Angle (1)

Critical for stability; prevents crashes.

Relative Position (3)

Distance to landing pad.

Relative Velocity (3)

Difference in speed for smooth touchdown.

*Action S*pace & Control

Drone controlled via 4 continuous actions

- Linear velocity in X, Y, Z
- Yaw (rotation)

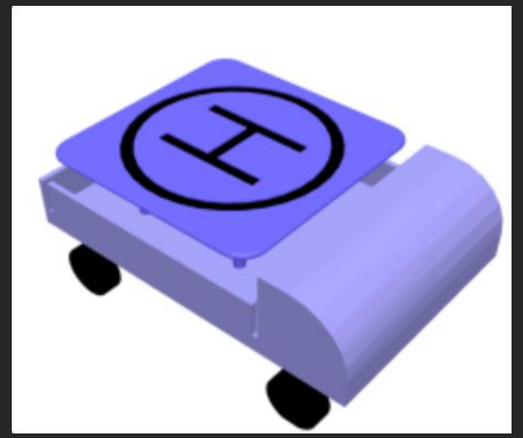
Kinetic overrides simulate realistic MuJoCo movement.

```
# Construct qvel vector
current_qvel = np.array([
    action[0], action[1], action[2], 0, action[3],
    0, 0, 0, 0, 0,
    car_vel_x,
    car_vel_y,
    0, 0, 0, 0, 0, 0, 0, 0, 0
])
```

```
def _get_obs(self):
    # Observation: Action (4) + Pitch (1) + Relative Vec (3) + relative velocity (3) = 11 values
    pitch = self.state_vector()[4]
    return np.concatenate([self.action_buffer, [pitch], self.rel_desired_heading_vec, self.rel_velocity_vec])
```

Target Dynamics: Moving Platform

Car movement uses Domain Randomization.



Forward Speed

Randomized between 0.25 and 0.29 m/s.

Lateral Sway

Sinusoidal movement with randomized amplitude and frequency.

Straight Line Logic

20% probability for basic convergence testing.

```python
# forward speed
self.rand_forward_speed = np.random.uniform(0.25, 0.29)

# Sway Width : 0.0 straight to 0.5 zigzag
self.rand_sway_amp = np.random.uniform(0.0, 0.5)

# Sway Speed : 0.01 gentle to 0.05 aggressive
self.rand_sway_freq = np.random.uniform(0.01, 0.05)

# Phase shift
self.rand_phase = np.random.uniform(0, 2 * np.pi)

# Straight line: 20% chance to a perfect straight line
if np.random.rand() < 0.2:
    self.rand_sway_amp = 0.0
```

Multi-Stage Reward Function

Guides drone through an "L-shaped" trajectory.

Distance Reward

Encourages centering and level flight.

Landing Reward

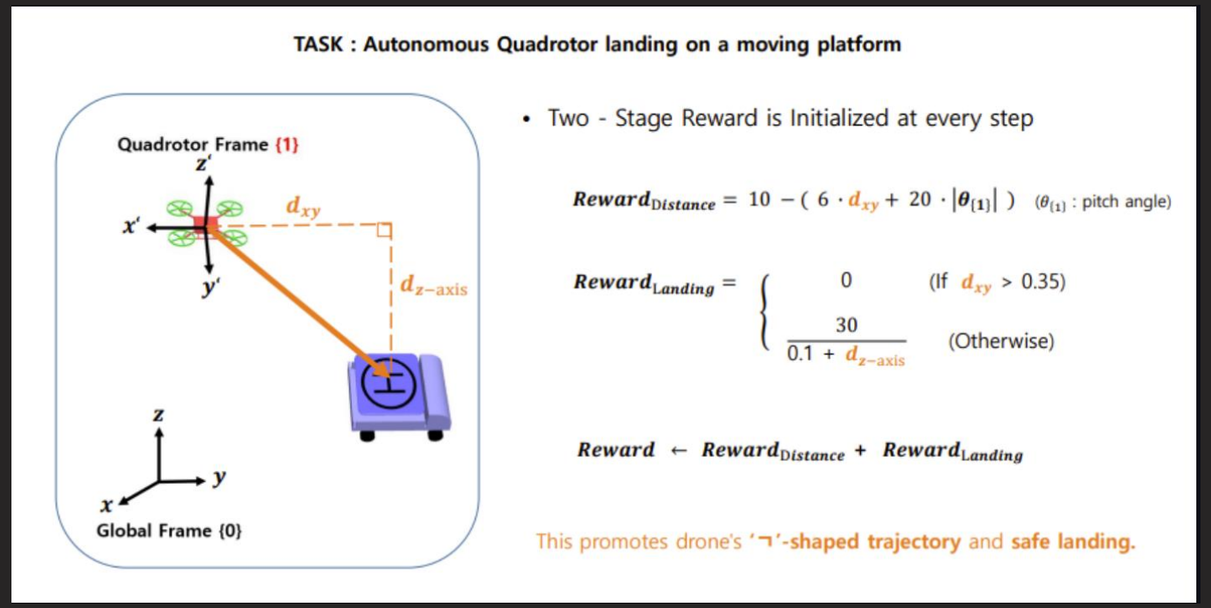
Activated when *z < 0.35*; bonus based on vertical proximity.

Smoothing Penalty

Penalties for sudden, jerky movements.

Terminal Success

Massive reward if all 4 landing gears touch simultaneously.



Safety Constraints & Reset Conditions

Simulation resets if any "unhealthy" state is reached.

Collision

Immediate reset if propellers hit car (-100 penalty).

Boundary

If drone drifts > 2 meters from target.

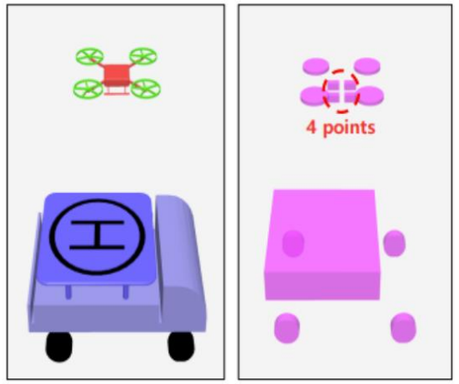
Instability

If pitch angle exceeds 0.8 radians.

Ground Impact

If drone hits floor instead of platform.

TASK : Autonomous Quadrotor landing on a moving platform



Visualization Hit Box

- If all 4 points (landing gear) of the drone hit the landing box ,
  $\boldsymbol{Reward}$ += $5 \cdot 10^5$
- if propeller hits car agent ,
  $\boldsymbol{Reward}$ -= 100
- Reward for overloading ,
  $\boldsymbol{Reward}$ -= $0.2 \cdot \|\text{action}_T - \text{action}_{T-1}\|$
- Reset condition
  If a **collision** occurs between the propeller and car agent ,
  If all 4 points of the drone hit the landing box ,
  If $d_{xy} > 3.0$ [m] ,
  If $\|$ *pitch angle* $\| > 0.8$ [rad] ,
  If drone hits the **ground** ,

The Control Challenge: Unpredictability

Dynamic Environment

Not a static landing pad

Physics simulation introduces significant randomness.

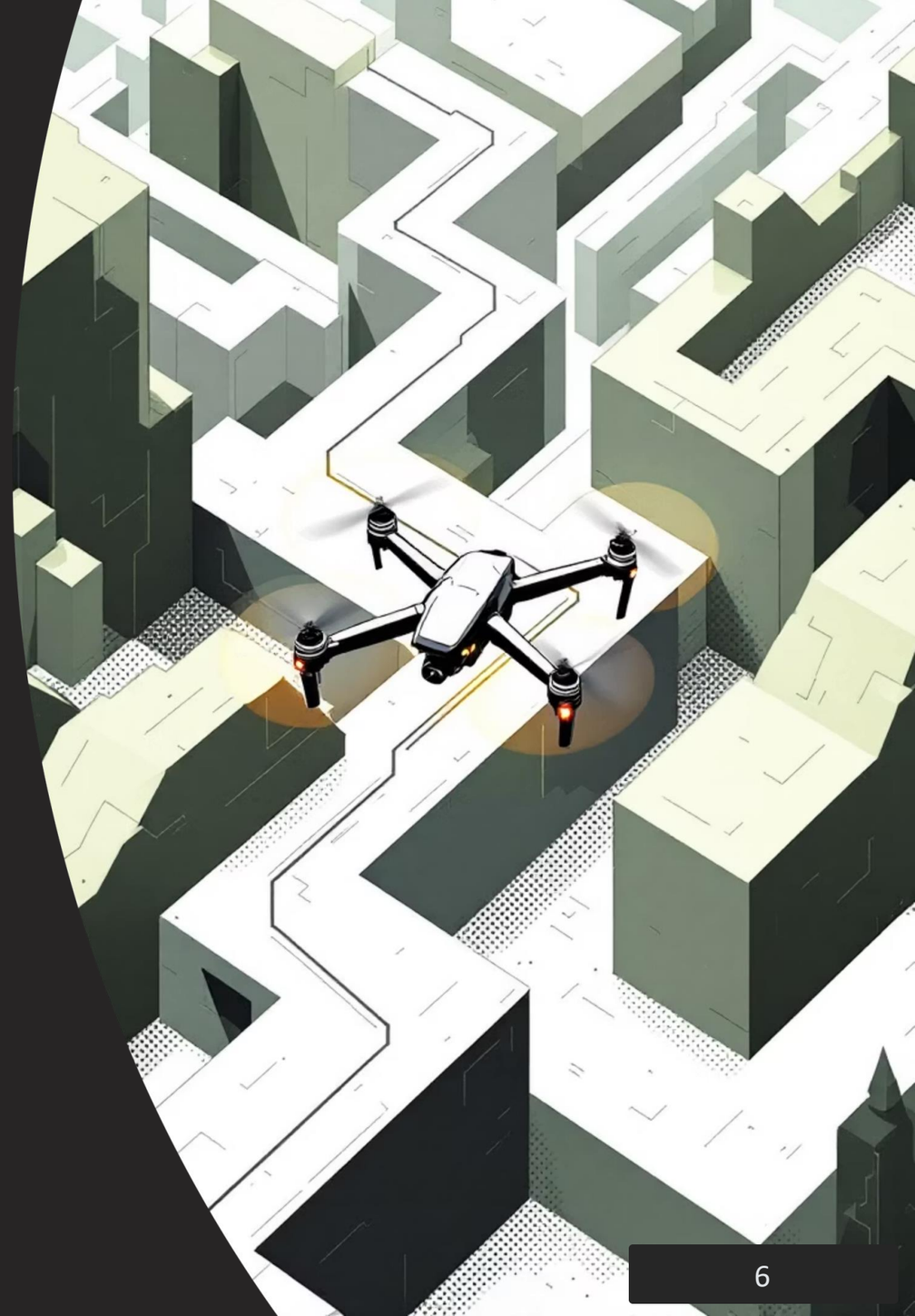
Variable Dynamics

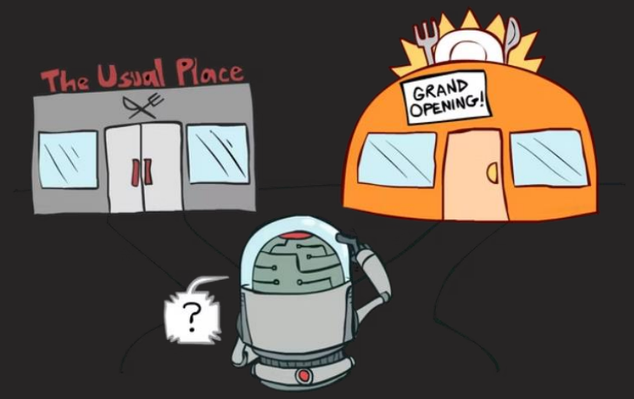
Target speed changes every episode (0.25 - 0.29 m/s).

Zig-Zag Adversary

Random oscillations prevent memorization of a single path.

A rigid policy would fail. We need a flexible solution.





Why SAC? The utility of "Soft"

Soft Actor-Critic (SAC)

Modern Off-Policy algorithm.

Key Innovation:

Entropy Maximization

$$J(\pi) = \sum_t \mathbb{E}[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))]$$

- Reward **r**: Gaining points (Landing).
- Entropy **H**: Keeping randomness in behavior.

Advantage for the Drone:

→ Forces exploration of diverse strategies.

→ Makes policy **robust** to perturbations (car's zig-zag).

→ Prevents premature convergence to bad solutions.

SAC Architecture

The Actor (Pilot)

- Input: State (11 values: pos, vel, pitch).
- Output: Probability distribution for 4 motors.

The Critic (Coach)

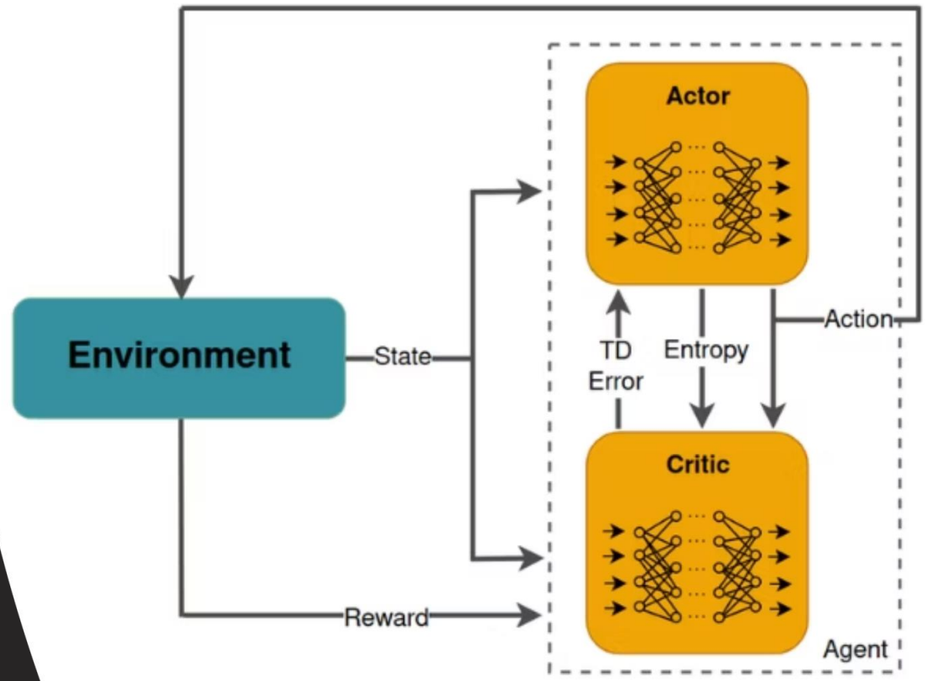
- Estimates action quality (Q-Value).
- Double Q-learning to avoid optimism.

Replay Buffer (Memory)

- Stores 1,000,000 past steps.
- Learns from previous crashes.

Auto-Tuning

Algorithm self-adjusts exploration level.



Conclusion: Why SAC Works

Continuous Space

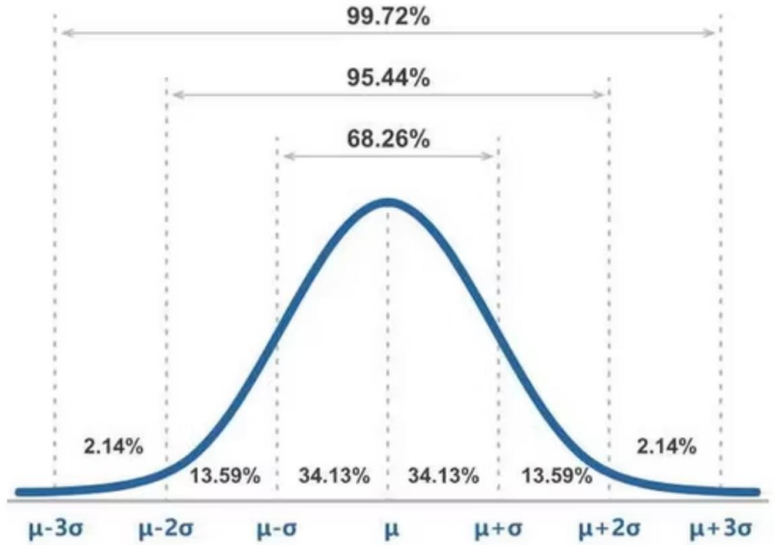
Perfect for fine motor thrust control (no ON/OFF).

Stability

Entropy stabilizes learning despite complex physics.

Performance

Consistently achieves Goal Reward (+500k).



PPO (Proximal Policy Optimization)

On-Policy Algorithm

Architecture:

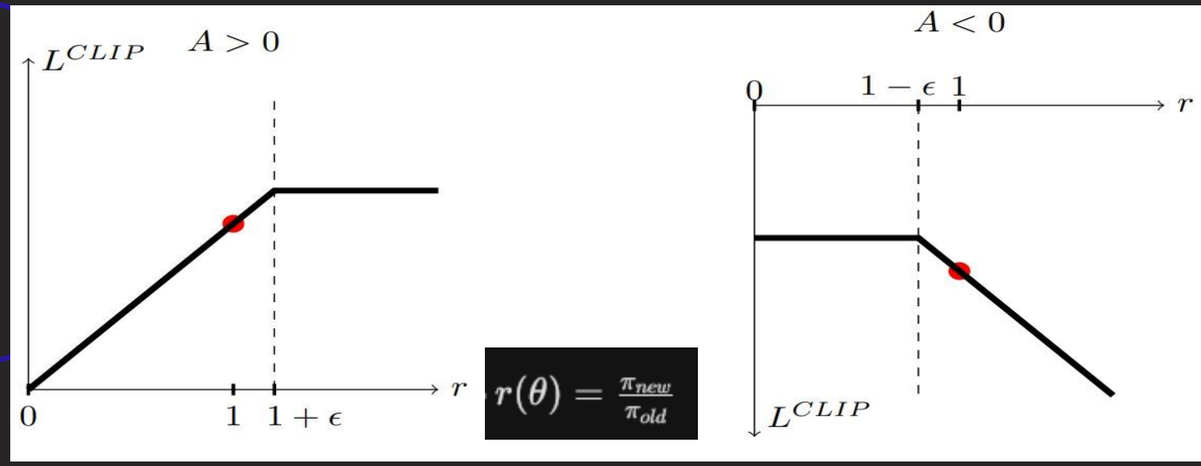
- **The policy network (πθ):** Maps observations to actions (Thrusts).
- **The value function network (Vϕ):** Estimates the value of the state to compute the **Advantage (At)**.

Core Mechanism:

Clipped Objective Function.

- *Prevents drastic policy updates.*
- *Ensures training stability*

Philosophy: "Conservative but Consistent" (vs. SAC's Maximum Entropy)



ϵ represents the clipping range (usually 0.2)

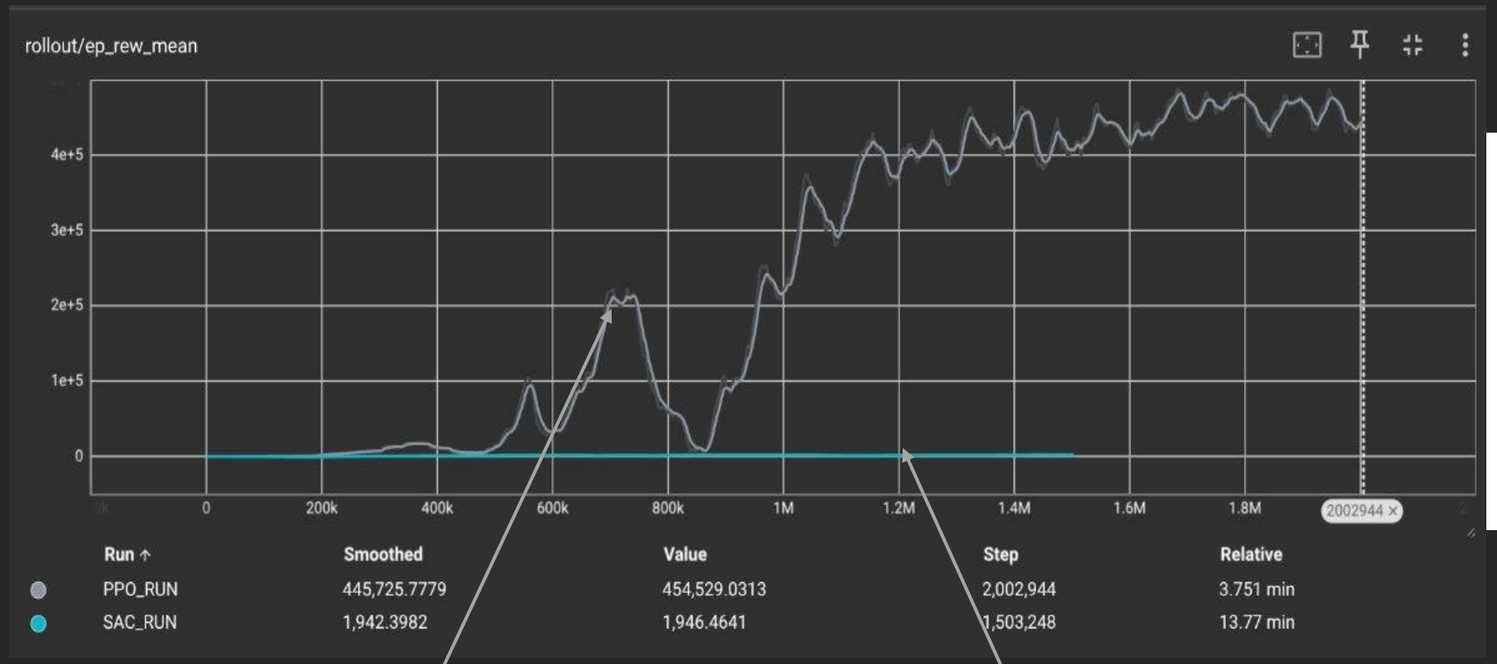
Quantitative Results (Efficiency & Success : SAC vs PPO)

Computational Efficiency:

- **PPO:** ~8,800 FPS (Fast & Lightweight).
- **SAC:** ~1,800 FPS (Computationally expensive).

Training Outcome:

- **SAC:** Flatline at reward ≈2,000 (Tracking only).
- **PPO:** Converged to ≈450,000 (Successful Landing).



Comparison Metric	PPO (Ours)	SAC
Algorithm Type	On-Policy	Off-Policy
Training Speed	≈ **8,800 FPS**	≈ 1,800 FPS
Max Reward	≈ **450,000**	≈ 2,000
Final Outcome	**Successful Landing**	Hovering / Timeout

Table 1: Performance summary: PPO vs. SAC

PPO Reward
Grey Curve

SAC Reward
Blue Curve

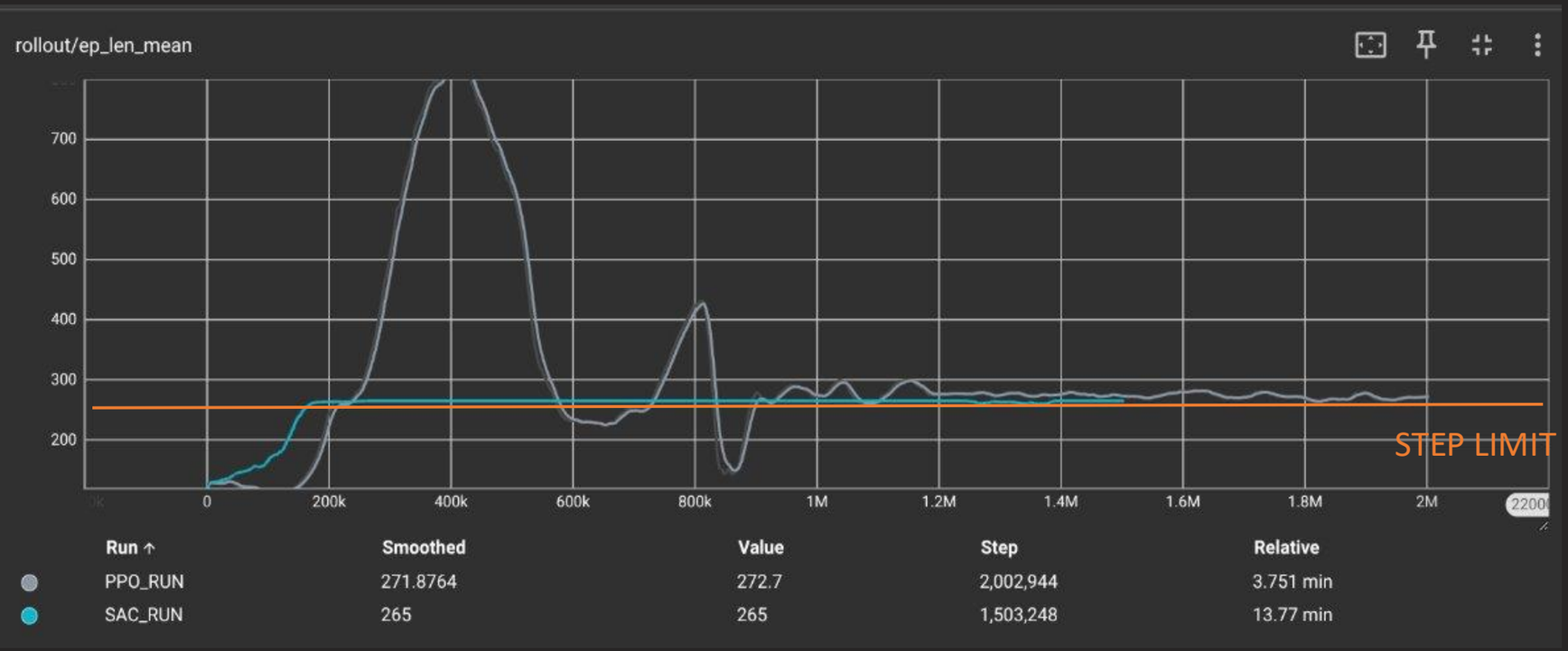
Behavioral Analysis (The "Safe Hovering" Trap)

Observation:

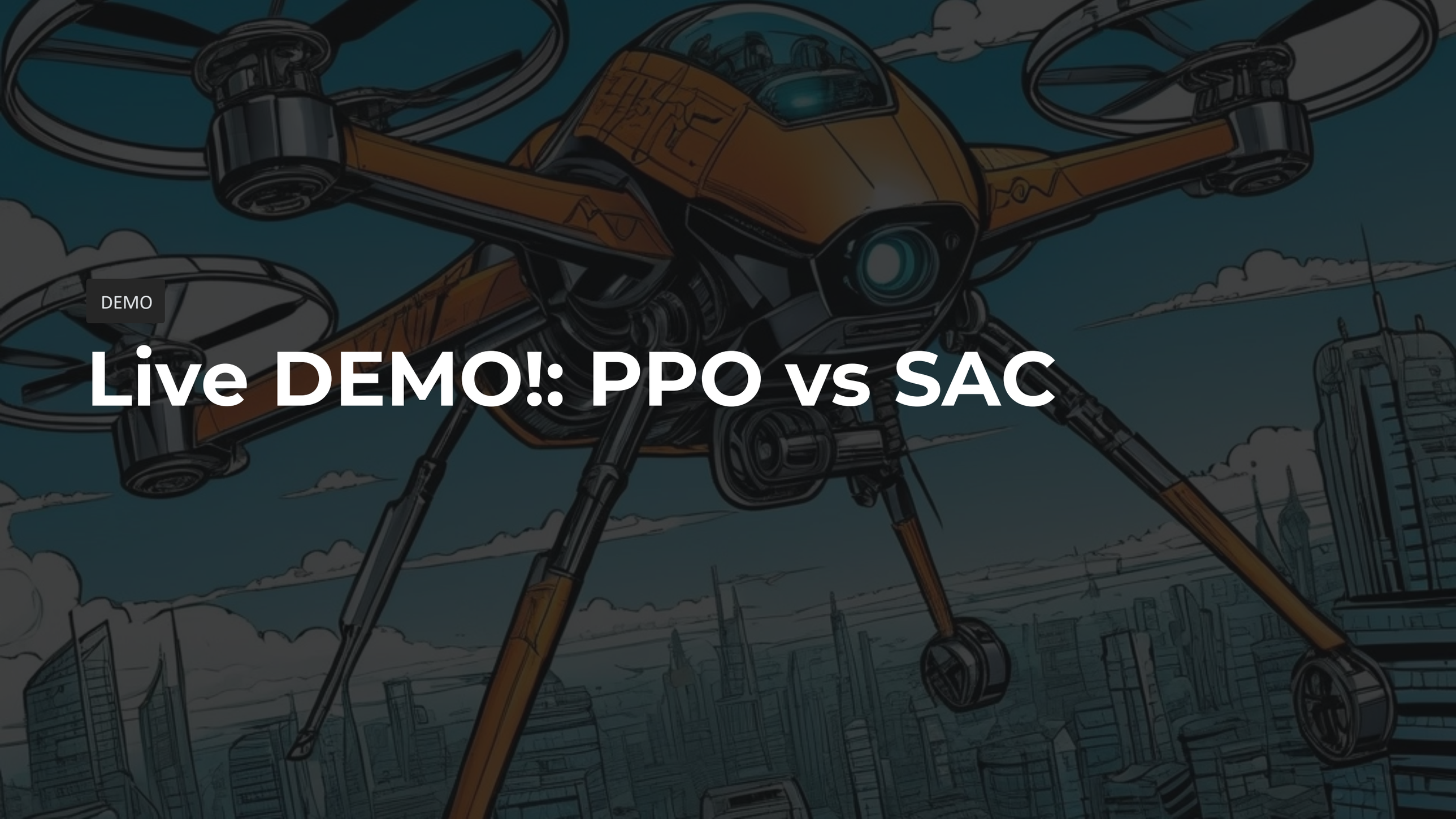
•SAC (Blue) consistently hits the artificial time limit (265 steps). It never lands.
•PPO (Grey) learns to terminate the episode efficiently upon landing.

The "Why" (Interpretation):

- SAC maximizes entropy and perceives landing as a high-risk action (crash penalty).
- **Conclusion:** SAC found a "local optimum": it prefers safe hovering to accumulate small tracking rewards rather than risking a landing.



Run ↑	Smoothed	Value	Step	Relative
PPO_RUN	271.8764	272.7	2,002,944	3.751 min
SAC_RUN	265	265	1,503,248	13.77 min



DEMO

Live DEMO!: PPO vs SAC